

Amazon RDS Enhanced Monitoring and CloudWatch Dashboard

To Begin with the Lab

Summary of the Lab

In this lab, the built-in monitoring capabilities of **Amazon RDS** were explored using the **Monitoring** tab and **Amazon CloudWatch**. Various database performance metrics such as CPU utilization, database connections, free storage space, free memory, read IOPS, write IOPS, and throughput were reviewed directly from the RDS console. A custom **CloudWatch Dashboard** was then created to visualize important RDS metrics across multiple database instances. Finally, a **CloudWatch Alarm** was configured to monitor CPU utilization and send email notifications through **Amazon SNS** whenever the defined threshold was exceeded. This demonstrated how AWS provides native database monitoring and alerting without requiring third-party tools.

Understanding Amazon RDS Monitoring and CloudWatch Alarms

Amazon RDS Monitoring provides real-time visibility into database performance and resource utilization. The monitoring data is collected by **Amazon CloudWatch**, allowing administrators to view metrics, create dashboards, and configure alerts. Instead of manually checking database health throughout the day, CloudWatch alarms can automatically notify administrators when predefined thresholds are crossed. This helps identify performance issues early and ensures proactive database management.

Example:

A production MySQL database normally operates at 20–30% CPU utilization. During a heavy reporting workload, CPU usage rises above 90% for more than five minutes. A CloudWatch alarm automatically sends an email notification to the database administrator through Amazon SNS, allowing the issue to be investigated before users experience performance degradation.

Lab Steps

Step 1 – Create any Database with Enhanced monitoring enabled

- I will create the Free Tier Database for lab purpose.
- Select **Aurora PostgreSQL** or **Amazon RDS PostgreSQL** based on the lab setup.

aws [Search] [Alt+S] United States (N. Virginia) cloudriteshk (463646775279) cf-shashank1

Aurora and RDS > Databases > Create database

Availability zone group [info](#)

Availability zone group
Select the availability zone where you want to create your database.

US East (N. Virginia)

Engine options

Engine type [info](#)

☐ Aurora (MySQL Compatible)
 ☐ Aurora (PostgreSQL Compatible)
 ☐ MySQL
 ☒ PostgreSQL

☐ MariaDB
 ☐ Oracle
 ☐ Microsoft SQL Server
 ☐ IBM Db2

Choose a database creation method

☒ Full configuration
You set all of the configuration options, including ones for availability, security, backups, and maintenance.
 ☐ Easy create
Use recommended best-practice configurations. Some configuration options can be changed after the database is created.

- Choose the current production version.

aws [Search] [Alt+S] United States (N. Virginia) cloudriteshk (463646775279) cf-shashank1

Aurora and RDS > Databases > Create database

Templates

Choose a sample template to meet your use case.

☐ Production
Use defaults for high availability and fast, consistent performance.
 ☐ Dev/Test
This instance is intended for development use outside of a production environment.
 ☒ Free tier
Use the RDS Free Tier to develop new applications, test existing applications, or gain hands-on experience with Amazon RDS. [Info](#)

Availability and durability

Deployment options [info](#)
Choose the deployment option that provides the availability and durability needed for your use case. AWS is committed to a certain level of uptime depending on the deployment option you choose. [Learn more about the Amazon RDS service-level agreement \(SLA\).](#)

☐ Multi-AZ DB cluster deployment (3 instances)
Creates a primary DB instance with two readable standbys in separate Availability Zones. This setup provides:
 • 99.95% uptime
 • Redundancy across Availability Zones
 • Increased read capacity
 • Reduced write latency

☐ Multi-AZ DB instance deployment (2 instances)
Creates a primary DB instance with a non-readable standby instance in a separate Availability Zone. This setup provides:
 • 99.95% uptime
 • Redundancy across Availability Zones

☒ Single-AZ DB instance deployment (1 instance)
Creates a single DB instance without standby instances. This setup provides:
 • 99.5% uptime
 • No data redundancy

- Set the database identifier.
- Configure the master username and password.
- Attach the required security group.
- Set **Public Access** to **Yes**.

Virtual private cloud (VPC) Info
Choose the VPC. The VPC defines the virtual networking environment for this DB instance.
Default VPC (vpc-099792461c7d0a616)
11 Subnets, 7 Availability Zones
After a database is created, you can't change its VPC. Only VPCs with a corresponding DB subnet group are listed.

DB subnet group Info
Choose the DB subnet group. The DB subnet group defines which subnets and IP ranges the DB instance can use in the VPC that you selected.
default-vpc-099792461c7d0a616
6 Subnets, 6 Availability Zones

Public access Info
☒ Yes
RDS assigns a public IP address to the database. Amazon EC2 instances and other resources outside of the VPC can connect to your database. Resources inside the VPC can also connect to the database. Choose one or more VPC security groups that specify which resources can connect to the database.
☐ No
RDS doesn't assign a public IP address to the database. Only Amazon EC2 instances and other resources inside the VPC can connect to your database. Choose one or more VPC security groups that specify which resources can connect to the database.

VPC security group (firewall) Info
Choose one or more VPC security groups to allow access to your database. Make sure that the security group rules allow the appropriate incoming traffic.
☒ Choose existing
Choose existing VPC security groups
☐ Create new
Create new VPC security group

Existing VPC security groups
Choose one or more options
RDS-SG X default X

- Change the monitoring time range.

Monitoring Info
Choose monitoring tools for this database. Database Insights provides a combined view of Performance Insights and Enhanced Monitoring for your fleet of databases. Database Insights pricing is separate from RDS monthly estimates. See [Amazon CloudWatch pricing](#).

☐ Database Insights - Advanced
• Retains 15 months of performance history
• Fleet-level monitoring
• Integration with CloudWatch Application Signals

☒ Database Insights - Standard
• Retains 7 days of performance history, with the option to pay for the retention of up to 24 months of performance history

Performance Insights
☒ Enable Performance Insights
With Performance Insights dashboard, you can visualize the database load on your Amazon RDS DB instance load and filter the load by waits, SQL statements, hosts, or users.

Retention period
7 days

AWS KMS key Info
(default) aws/rds

Account
463646775279

KMS key ID
fd9c9c8d-03ce-425a-917f-1d2e959cf13e

You can't change the KMS key after you create your database.

- Enable **Enhanced Monitoring** and choose **Metric Granularity** to 30 seconds for this lab.

Performance Insights

☐ Enable Performance Insights
With Performance Insights dashboard, you can visualize the database load on your Amazon RDS DB instance load and filter the load by waits, SQL statements, hosts, or users.

▼ **Additional monitoring settings**
Enhanced Monitoring, CloudWatch Logs and DevOps Guru

Enhanced Monitoring

☒ Enable Enhanced monitoring
Enabling Enhanced Monitoring metrics are useful when you want to see how different processes or threads use the CPU.

OS metrics granularity

30 seconds

Monitoring role for OS metrics

default

The monitoring role is an IAM role that allows RDS to send Enhanced Monitoring metrics to Amazon CloudWatch Logs. Choose an existing monitoring role, or choose default to have RDS automatically create the IAM role `rds-monitoring-role` for you.

[Create new role](#)

Log exports
Select the log types to publish to Amazon CloudWatch Logs

☒ iam-db-auth-error log
☒ PostgreSQL log
☒ Upgrade log

IAM role
The following service-linked role is used for publishing logs to CloudWatch Logs.

RDS service-linked role

- Create the database and wait for it to become available.

Step 2 – Connect to the Database

- On the **Connectivity & security** panel connect using Code Snippets and run the commands in Command Prompt to connect to database.

Aurora and RDS > **Databases** > database-1

Connectivity & security | Monitoring | Logs & events | Configuration | Zero-ETL integrations | Maintenance & backups | Data migrations | Tags

Connect using Info

☒ Code snippets
Connect using SDKs, APIs, or third-party tools including agents.

☐ CloudShell
Use for a quick access to AWS CLI that launches directly from the AWS Management Console.

☐ Endpoints
Use when connecting through any IDE interface.

Internet access gateway
Disabled

IAM Authentication
Disabled

Programming language | **Endpoint type**

PSQL (Windows) | Instance endpoint

Connection steps
Follow the steps below to create the code of each step to use and run the commands. The commands are provided as a reference only.

```
1 | curl -o global-bundle.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
```

PowerShell

```
1 | psql "host=database-1.cghaxz7pag8a.us-east-1.rds.amazonaws.com port=5432 dbname=postgres user=postgres sslmode=verify-full sslrootcert=./global-bundle.pem"
```

Connected compute resources (0) Info

Connections to compute resources that were created automatically by RDS are shown here. Connections to compute resources that were created manually aren't shown.

[Filter by compute resources](#)

- Enter your password and you are connected to database.

```

Command Prompt - psql "ho" X + v
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Shashank>curl -o global-bundle.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 161.5k 100 161.5k 0 0 167.6k 0 0 0 0 0

C:\Users\Shashank>psql "host=database-1.cghaxz7pqg8a.us-east-1.rds.amazonaws.com port=5432 dbname=postgres user=postgres
sslmode=verify-full sslrootcert=./global-bundle.pem"
Password for user postgres:

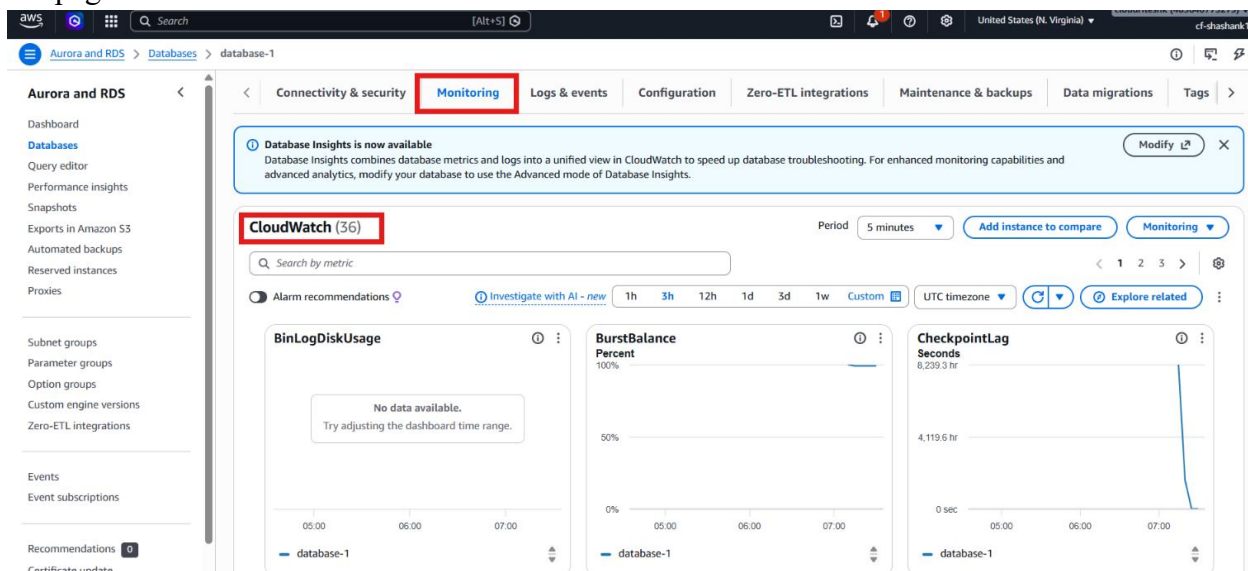
psql (18.2)
WARNING: Console code page (437) differs from Windows code page (1252)
8-bit characters might not work correctly. See psql reference
page "Notes for Windows users" for details.
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
Type "help" for help.

postgres=> |

```

Step 3 – Create tables and Observe Metrics

- After connecting to table, ran the following commands and see the metrics on **Monitoring** page.



- Create tables and insert the values for lab purpose.

```

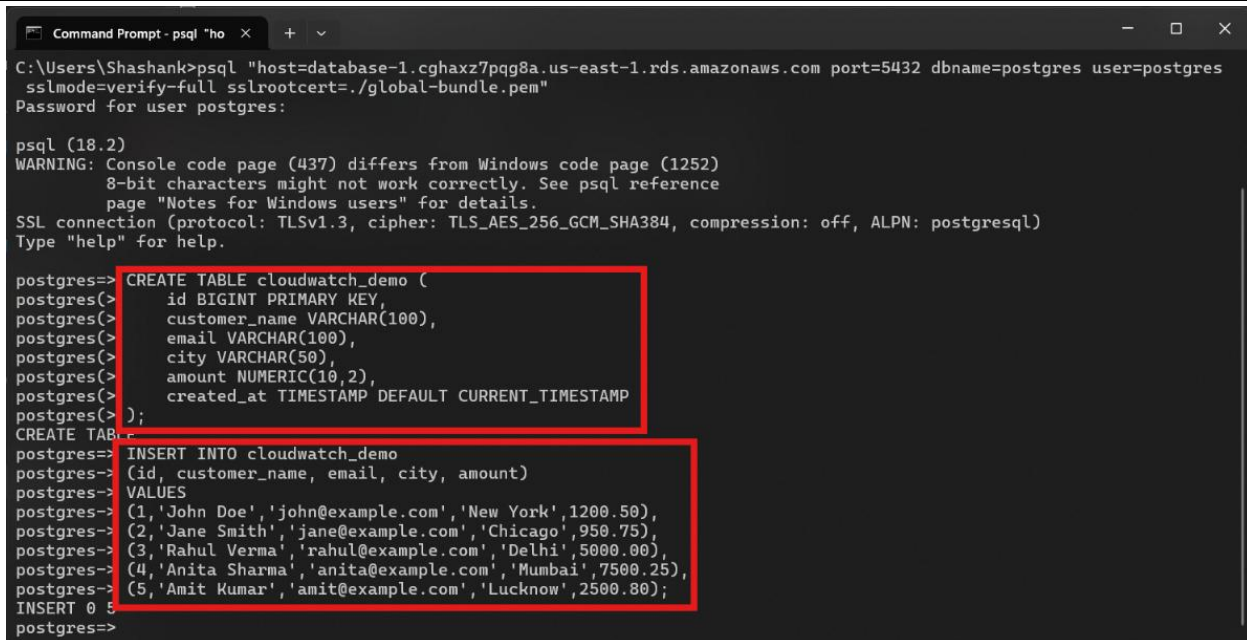
CREATE TABLE cloudwatch_demo (
  id BIGINT PRIMARY KEY,
  customer_name VARCHAR(100),
  email VARCHAR(100),
  city VARCHAR(50),
  amount NUMERIC(10,2),

```

```
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

- Insert data

```
INSERT INTO cloudwatch_demo
(id, customer_name, email, city, amount)
VALUES
(1,'John Doe','john@example.com','New York',1200.50),
(2,'Jane Smith','jane@example.com','Chicago',950.75),
(3,'Rahul Verma','rahul@example.com','Delhi',5000.00),
(4,'Anita Sharma','anita@example.com','Mumbai',7500.25),
(5,'Amit Kumar','amit@example.com','Lucknow',2500.80);
```



```
Command Prompt - psql 'ho ...
C:\Users\Shashank>psql "host=database-1.cghaxz7pqq8a.us-east-1.rds.amazonaws.com port=5432 dbname=postgres user=postgres
sslmode=verify-full sslrootcert=./global-bundle.pem"
Password for user postgres:

psql (18.2)
WARNING: Console code page (437) differs from Windows code page (1252)
8-bit characters might not work correctly. See psql reference
page "Notes for Windows users" for details.
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
Type "help" for help.

postgres=> CREATE TABLE cloudwatch_demo (
postgres=>     id BIGINT PRIMARY KEY,
postgres=>     customer_name VARCHAR(100),
postgres=>     email VARCHAR(100),
postgres=>     city VARCHAR(50),
postgres=>     amount NUMERIC(10,2),
postgres=>     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
postgres=> );
CREATE TABLE
postgres=> INSERT INTO cloudwatch_demo
postgres=> (id, customer_name, email, city, amount)
postgres=> VALUES
postgres=> (1,'John Doe','john@example.com','New York',1200.50),
postgres=> (2,'Jane Smith','jane@example.com','Chicago',950.75),
postgres=> (3,'Rahul Verma','rahul@example.com','Delhi',5000.00),
postgres=> (4,'Anita Sharma','anita@example.com','Mumbai',7500.25),
postgres=> (5,'Amit Kumar','amit@example.com','Lucknow',2500.80);
INSERT 0 5
postgres=>
```

- Now for monitoring the values, we will generate high load with some activities and then observe it on CloudWatch.
- For this we will generate huge rows and then perform read and update activity multiple times.
- Generate values for table.

```
INSERT INTO cloudwatch_demo
SELECT
gs,
'Customer_' || gs,
'customer' || gs || '@example.com',
'City_' || (gs % 100),
(random() * 10000)::numeric(10,2)
```

```
FROM generate_series(1000,100999) gs;
```

```
Command Prompt - psql "ho" X + v
CREATE TABLE
postgres=> INSERT INTO cloudwatch_demo
postgres-> (id, customer_name, email, city, amount)
postgres-> VALUES
postgres-> (1,'John Doe','john@example.com','New York',1200.50),
postgres-> (2,'Jane Smith','jane@example.com','Chicago',950.75),
postgres-> (3,'Rahul Verma','rahul@example.com','Delhi',5000.00),
postgres-> (4,'Anita Sharma','anita@example.com','Mumbai',7500.25),
postgres-> (5,'Amit Kumar','amit@example.com','Lucknow',2500.80);
INSERT 0 5
postgres=> INSERT INTO cloudwatch_demo
postgres-> SELECT
postgres->     gs,
postgres->     'Customer_' || gs
postgres->     'customer' || gs || '@example.com',
postgres->     'City_' || (gs % 100),
postgres->     (random() * 10000)::numeric(10,2)
postgres-> FROM generate_series(1000,100999) gs;
INSERT 0 100000
postgres=> ELECT COUNT(*) FROM cloudwatch_demo;
ERROR:  syntax error at or near "ELECT"
LINE 1: ELECT COUNT(*) FROM cloudwatch_demo;
        ^
postgres=>
postgres=> SELECT city, COUNT(*)
postgres-> FROM cloudwatch_demo
postgres-> GROUP BY city;
  city  | count
-----+-----
City_62 | 1000
```

- Generate Read Activity.

```
SELECT COUNT(*) FROM cloudwatch_demo;
```

```
SELECT city, COUNT(*)
FROM cloudwatch_demo
GROUP BY city;
```

```
SELECT *
FROM cloudwatch_demo
ORDER BY amount DESC
LIMIT 100;
```

```
Command Prompt - psql "ho" x + v
City_84 | 1000
postgres=# SELECT COUNT(*) FROM cloudwatch_demo;
count
-----
100005
(1 row)

postgres=#
postgres=# SELECT city, COUNT(*)
postgres=# FROM cloudwatch_demo
postgres=# GROUP BY city;
 city | count
-----+-----
City_62 | 1000
City_15 | 1000
City_90 | 1000
City_16 | 1000
Lucknow | 1
City_96 | 1000
City_72 | 1000
City_50 | 1000
City_47 | 1000
City_56 | 1000
City_17 | 1000
City_54 | 1000
City_35 | 1000
City_77 | 1000
City_64 | 1000
City_52 | 1000
```

- Generate Update Activity.

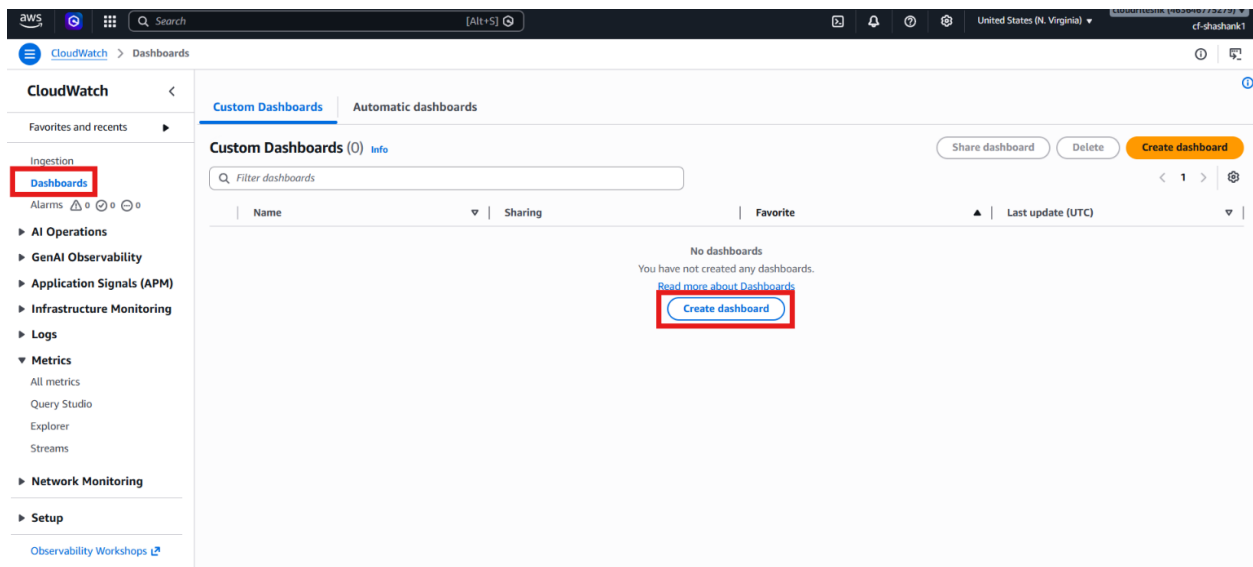
```
UPDATE cloudwatch_demo
SET amount = amount + 100
WHERE id % 2 = 0;
```

```
Command Prompt - psql "ho" x + v
City_65 | 1000
City_86 | 1000
City_19 | 1000
City_0 | 1000
City_8 | 1000
New York | 1
City_81 | 1000
City_25 | 1000
City_61 | 1000
City_31 | 1000
City_87 | 1000
City_79 | 1000
City_59 | 1000
City_22 | 1000
City_91 | 1000
City_82 | 1000
City_48 | 1000
City_41 | 1000
City_24 | 1000
City_20 | 1000
City_3 | 1000
City_92 | 1000
City_73 | 1000
City_71 | 1000
City_84 | 1000
postgres=# UPDATE cloudwatch_demo
postgres=# SET amount = amount + 100
postgres=# WHERE id % 2 = 0;
UPDATE 50002
postgres=#
```

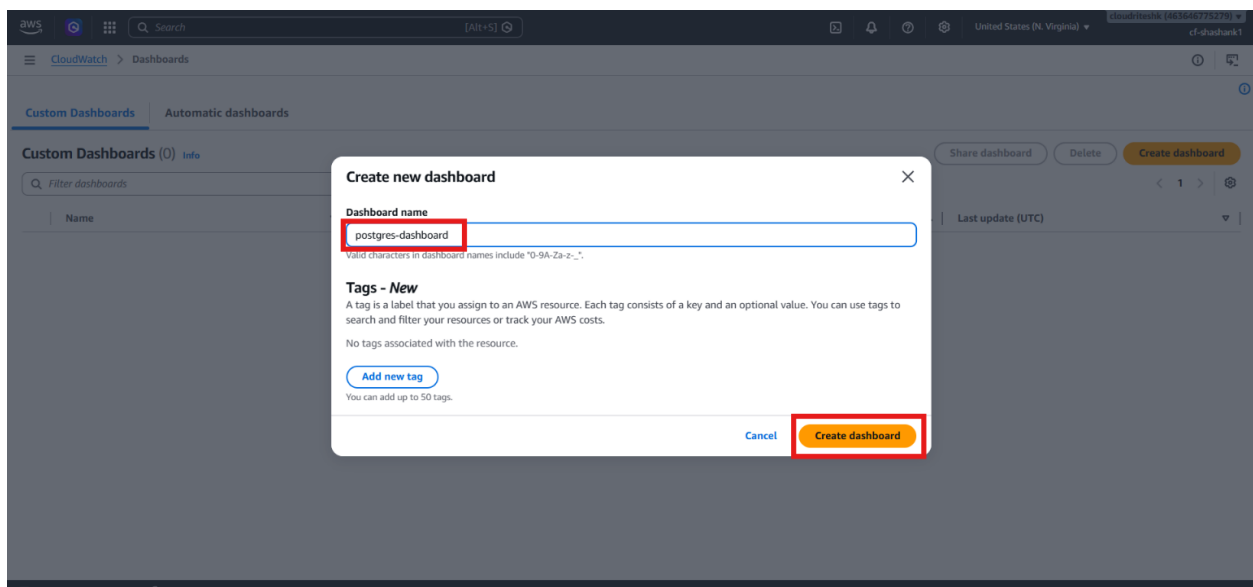
Step 4 – Open Amazon CloudWatch and Create Custom Dashboard

- Navigate to **Amazon CloudWatch**.
- Open the **CloudWatch Dashboard** homepage.
- From the left navigation panel, select **Dashboards**.

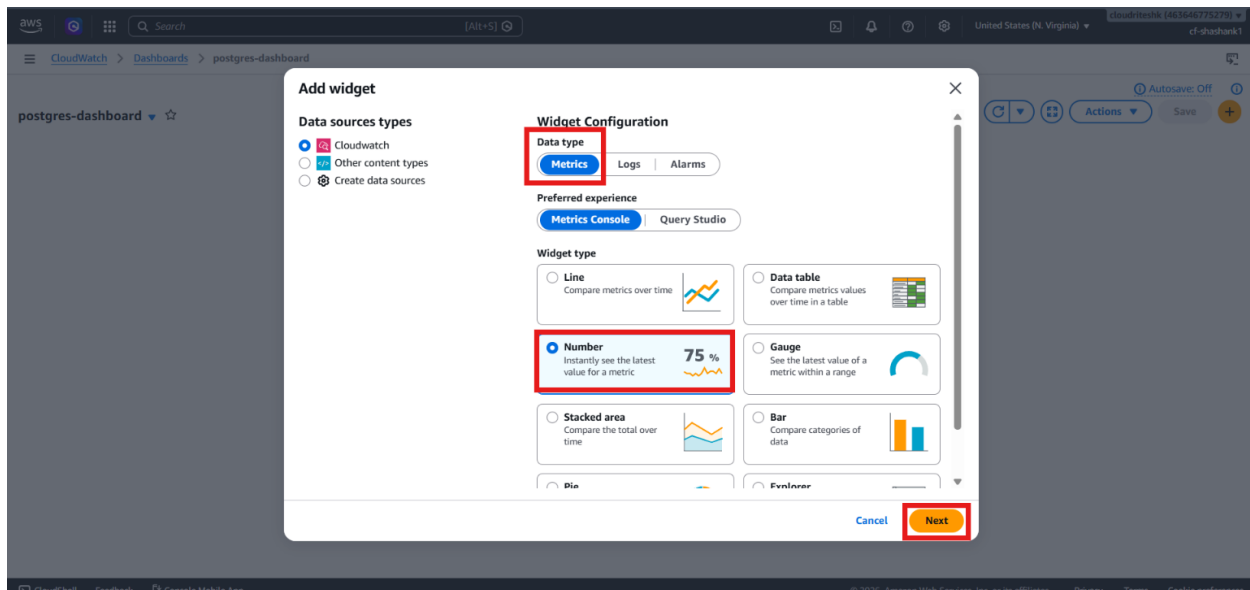
- Click **Create Dashboard**.



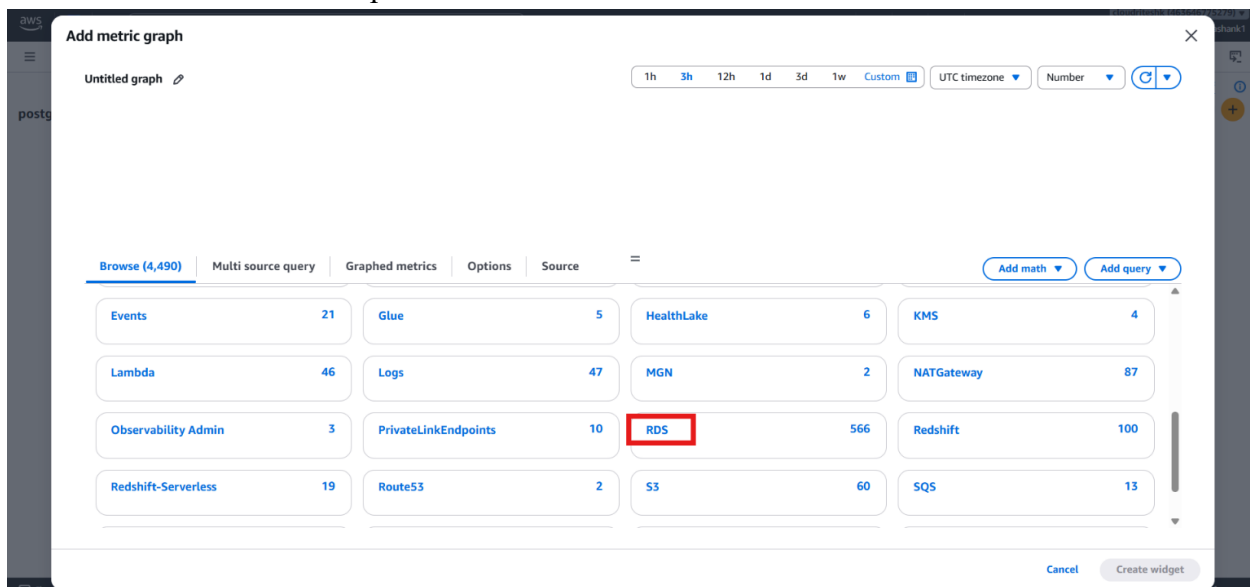
- Enter a dashboard name.
- Click **Create Dashboard**.



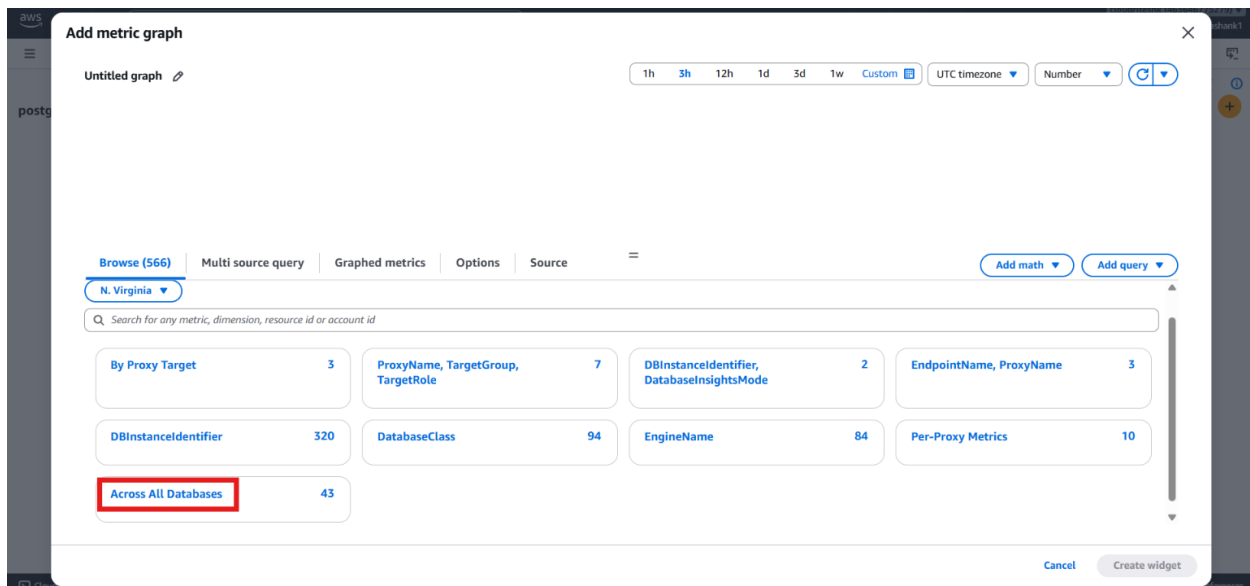
- Select the **Explorer** widget.



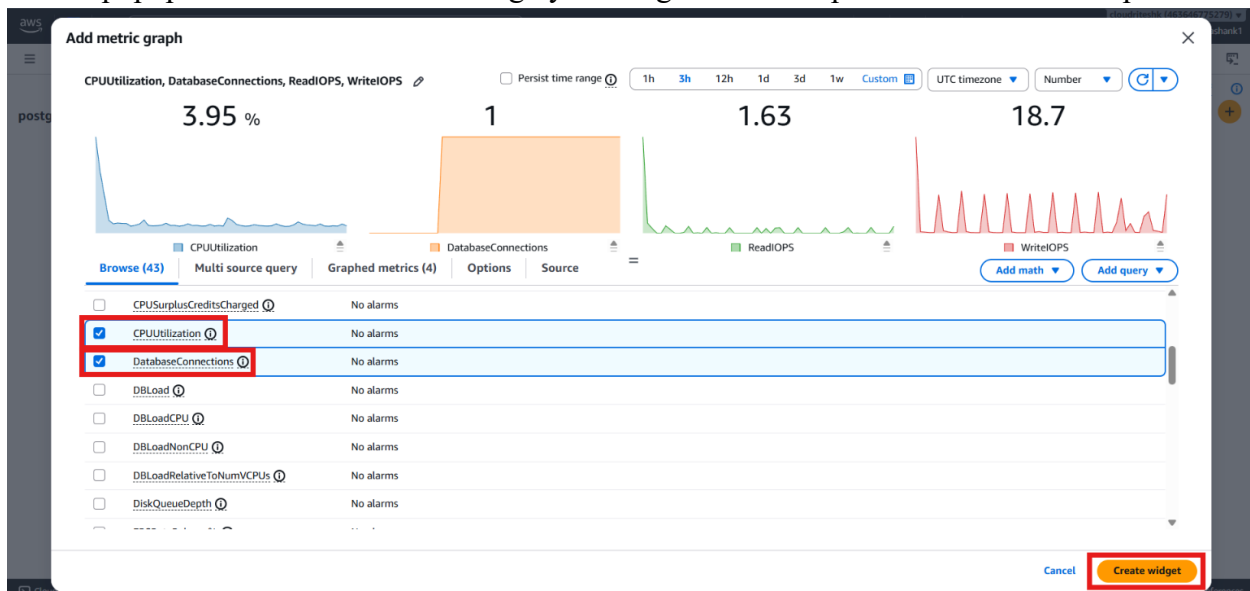
- Choose the **Data & Widget type**.
- Select RDS from Namespace.



- Select Across All database.



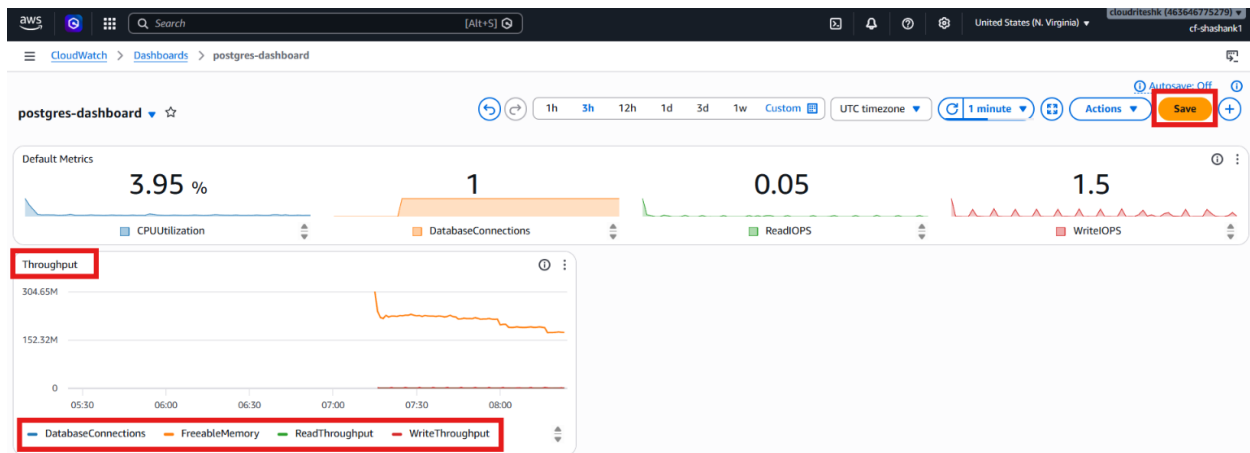
- Choose the following: CPUUtilization, DatabaseConnections, ReadIOPS, WriteIOPS, from the popup and monitor the following by running Read and Update commands multiple times.



- And click **Create**.

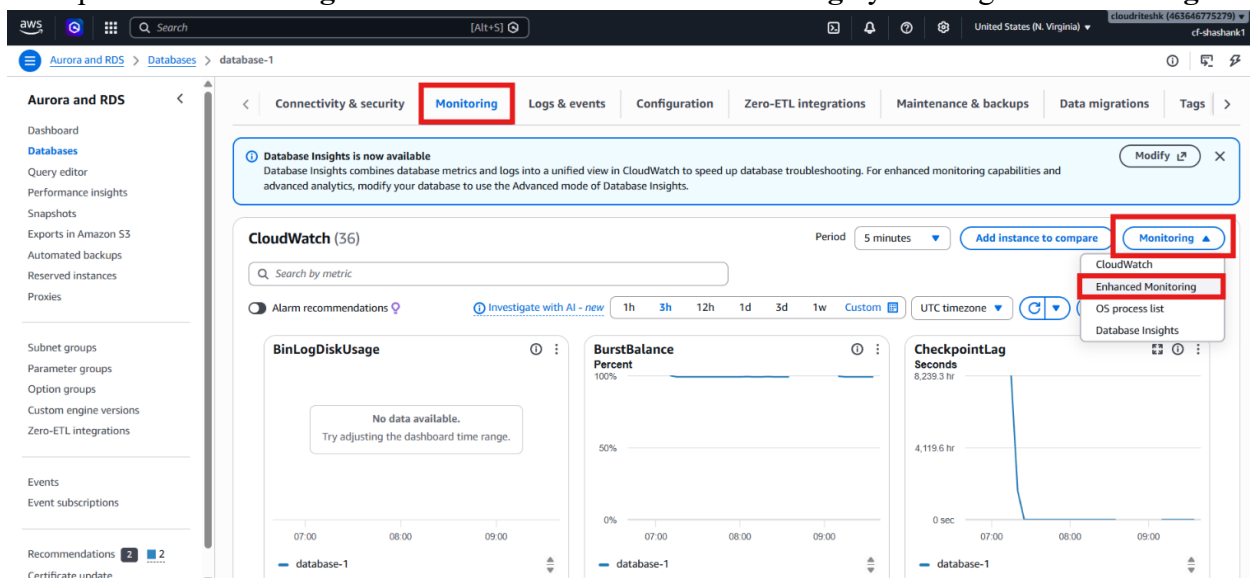
Step 5 – Add RDS Metrics to the Dashboard

- Open the **Metrics** section.
- Add more metrics like this if you want to customize the dashboard:
 - CPU Utilization
 - Database Connections
- Choose name for the widget and save.
- Click **Save Dashboard**.

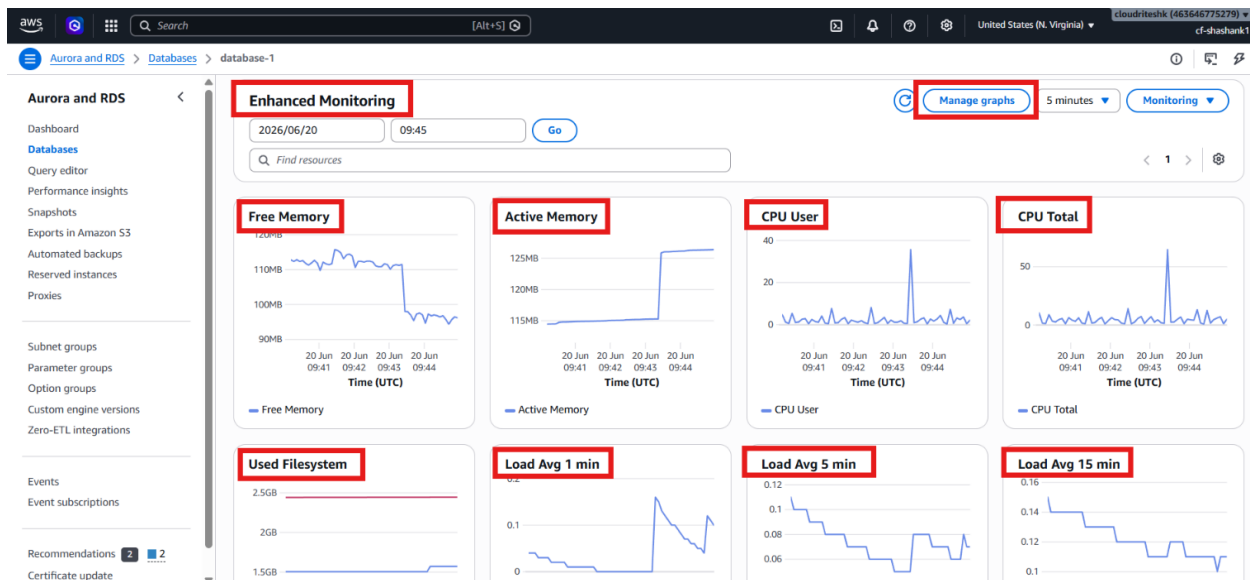


Step 6 – Review the Enhanced Monitoring Panel

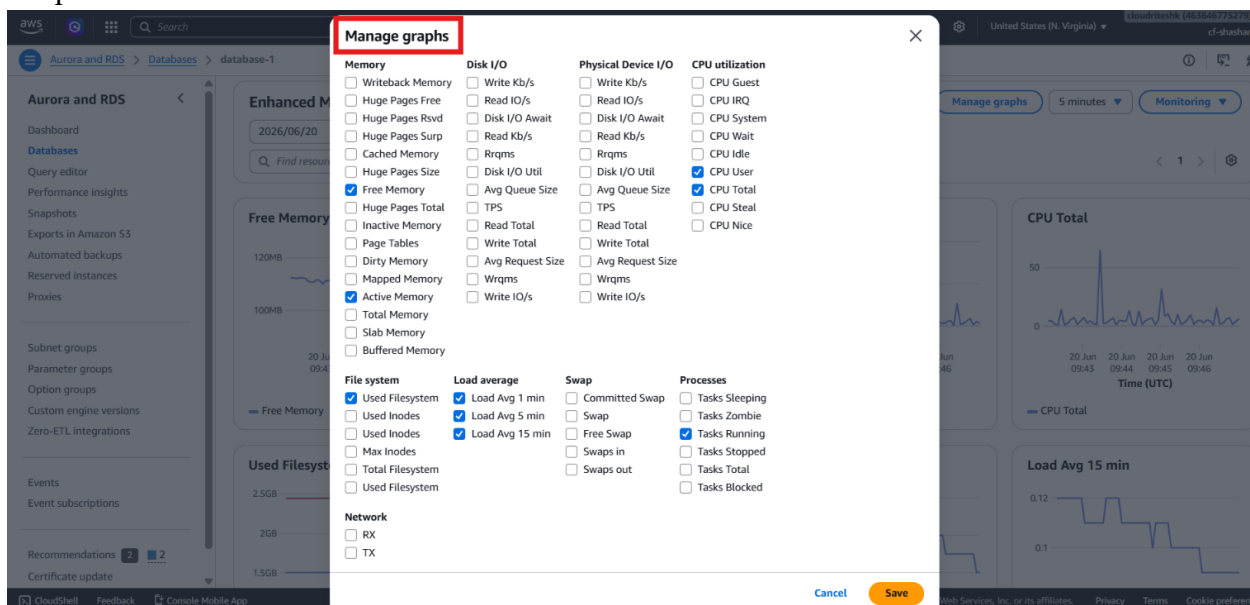
- Open the **Monitoring** tab and choose **Enhanced Monitoring** by clicking on **Monitoring**.



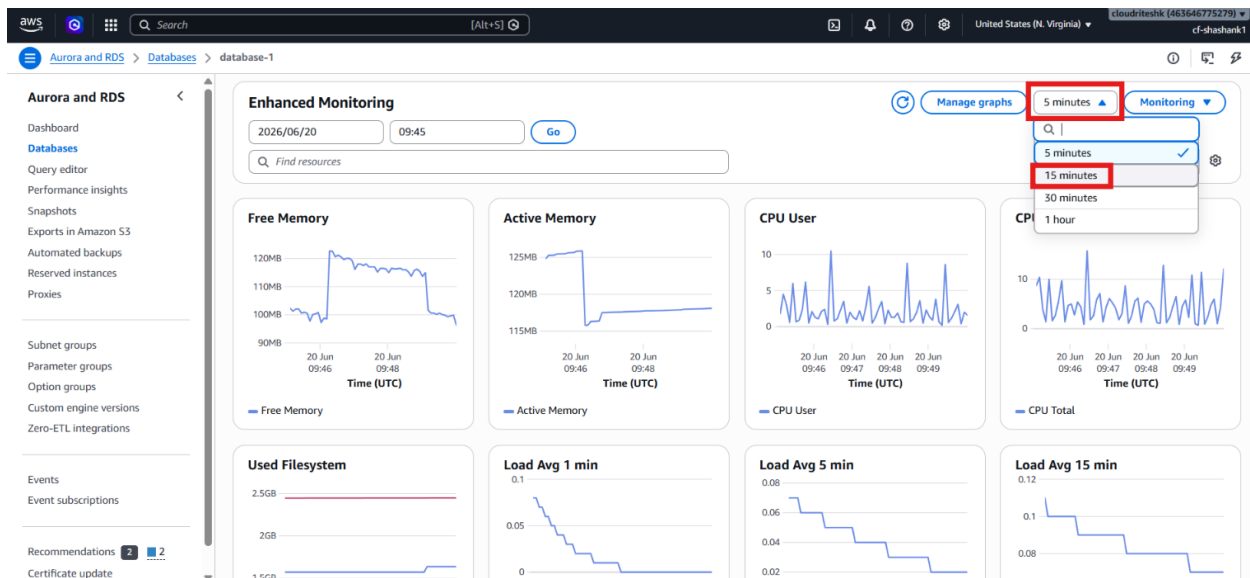
- New enhanced RDS metrics appear like Free Memory, Active Memory, CPU User, CPU Total, etc.



- Click on Manage Graph to see various options and customize metrics on your own preference.



- Choose your desired graphs and click **Save**.
- You can also choose Desired time interval for watching the metrics as well by clicking on 5 Minutes.



- Review historical and real-time database performance information.

Step 7 – Review the Enhanced Monitoring JSON Logs

- Open CloudWatch and click on Log management on left panel and click on **RDSOSMetrics**.

- Click on your log stream.

The screenshot shows the AWS CloudWatch console interface. On the left is a navigation sidebar with categories like Ingestion, Dashboards, Alarms, AI Operations, GenAI Observability, Application Signals (APM), Infrastructure Monitoring, Logs, Metrics, and Streams. The main content area is titled 'Log management' and 'RDSOSMetrics'. It displays details for a log group, including its ARN, creation time, retention, and various filters. The 'Log streams' tab is active, showing a list of log streams. The stream 'db-SQ3UGNWJX4C2JEMH5C2VUGCMPY' is highlighted with a red box, showing its last event time as 2026-06-20 13:04:39 (UTC).

- Now we can see our logs getting generated in JSON format.

This screenshot shows the 'Log events' view within the CloudWatch console. The breadcrumb navigation indicates the path: CloudWatch > Log management > RDSOSMetrics > db-SQ3UGNWJX4C2JEMH5C2VUGCMPY. The 'Log events' tab is selected and highlighted with a red box. Below the tab, there's a search bar and filters for time range (1m, 30m, 1h, 12h, Custom), UTC timezone, and display options. A table of log events is shown with columns for 'Timestamp' and 'Message'. The first event, with timestamp '2026-06-20T13:15:39.000Z', is highlighted with a red box. The message is a JSON object: {"engine": "POSTGRES", "instanceID": "database-1", "instanceResourceID": "db-SQ3UGNWJX4C2JEMH5C2VUGCMPY", "timestamp": "2026-06-20T13:15:39Z", "version": "1", "uptime": "..."}. At the bottom, a status message says 'No newer events at this moment. Auto retry paused. Resume'.

- And now if you see closely on timestamps, you can see the logs getting generated every 30 seconds.

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Filter events - press enter to search

Timestamp	Message
2026-06-20T13:10:39.000Z	{"engine":"POSTGRES","instanceID":"database-1","instanceResourceID":"db-SQUGNWX4C2JEMH5CZVUGCMFY","timestamp":"2026-06-20T13:10:39Z","version":1,"uptime":1.0}
2026-06-20T13:11:09.000Z	{"engine":"POSTGRES","instanceID":"database-1","instanceResourceID":"db-SQUGNWX4C2JEMH5CZVUGCMFY","timestamp":"2026-06-20T13:11:09Z","version":1,"uptime":1.0}
2026-06-20T13:11:39.000Z	{"engine":"POSTGRES","instanceID":"database-1","instanceResourceID":"db-SQUGNWX4C2JEMH5CZVUGCMFY","timestamp":"2026-06-20T13:11:39Z","version":1,"uptime":1.0}
2026-06-20T13:12:09.000Z	{"engine":"POSTGRES","instanceID":"database-1","instanceResourceID":"db-SQUGNWX4C2JEMH5CZVUGCMFY","timestamp":"2026-06-20T13:12:09Z","version":1,"uptime":1.0}
2026-06-20T13:12:39.000Z	{"engine":"POSTGRES","instanceID":"database-1","instanceResourceID":"db-SQUGNWX4C2JEMH5CZVUGCMFY","timestamp":"2026-06-20T13:12:39Z","version":1,"uptime":1.0}
2026-06-20T13:13:09.000Z	{"engine":"POSTGRES","instanceID":"database-1","instanceResourceID":"db-SQUGNWX4C2JEMH5CZVUGCMFY","timestamp":"2026-06-20T13:13:09Z","version":1,"uptime":1.0}
2026-06-20T13:13:39.000Z	{"engine":"POSTGRES","instanceID":"database-1","instanceResourceID":"db-SQUGNWX4C2JEMH5CZVUGCMFY","timestamp":"2026-06-20T13:13:39Z","version":1,"uptime":1.0}
2026-06-20T13:14:09.000Z	{"engine":"POSTGRES","instanceID":"database-1","instanceResourceID":"db-SQUGNWX4C2JEMH5CZVUGCMFY","timestamp":"2026-06-20T13:14:09Z","version":1,"uptime":1.0}
2026-06-20T13:14:39.000Z	{"engine":"POSTGRES","instanceID":"database-1","instanceResourceID":"db-SQUGNWX4C2JEMH5CZVUGCMFY","timestamp":"2026-06-20T13:14:39Z","version":1,"uptime":1.0}
2026-06-20T13:15:09.000Z	{"engine":"POSTGRES","instanceID":"database-1","instanceResourceID":"db-SQUGNWX4C2JEMH5CZVUGCMFY","timestamp":"2026-06-20T13:15:09Z","version":1,"uptime":1.0}
2026-06-20T13:15:39.000Z	{"engine":"POSTGRES","instanceID":"database-1","instanceResourceID":"db-SQUGNWX4C2JEMH5CZVUGCMFY","timestamp":"2026-06-20T13:15:39Z","version":1,"uptime":1.0}
2026-06-20T13:16:09.000Z	{"engine":"POSTGRES","instanceID":"database-1","instanceResourceID":"db-SQUGNWX4C2JEMH5CZVUGCMFY","timestamp":"2026-06-20T13:16:09Z","version":1,"uptime":1.0}
2026-06-20T13:16:39.000Z	{"engine":"POSTGRES","instanceID":"database-1","instanceResourceID":"db-SQUGNWX4C2JEMH5CZVUGCMFY","timestamp":"2026-06-20T13:16:39Z","version":1,"uptime":1.0}
2026-06-20T13:17:09.000Z	{"engine":"POSTGRES","instanceID":"database-1","instanceResourceID":"db-SQUGNWX4C2JEMH5CZVUGCMFY","timestamp":"2026-06-20T13:17:09Z","version":1,"uptime":1.0}
2026-06-20T13:17:39.000Z	{"engine":"POSTGRES","instanceID":"database-1","instanceResourceID":"db-SQUGNWX4C2JEMH5CZVUGCMFY","timestamp":"2026-06-20T13:17:39Z","version":1,"uptime":1.0}

No newer events at this moment. Auto retry paused. [Resume](#)

[Back to top](#)

- Now we will copy it and paste in on online JSON viewer to see the groups and data for easy understanding.

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

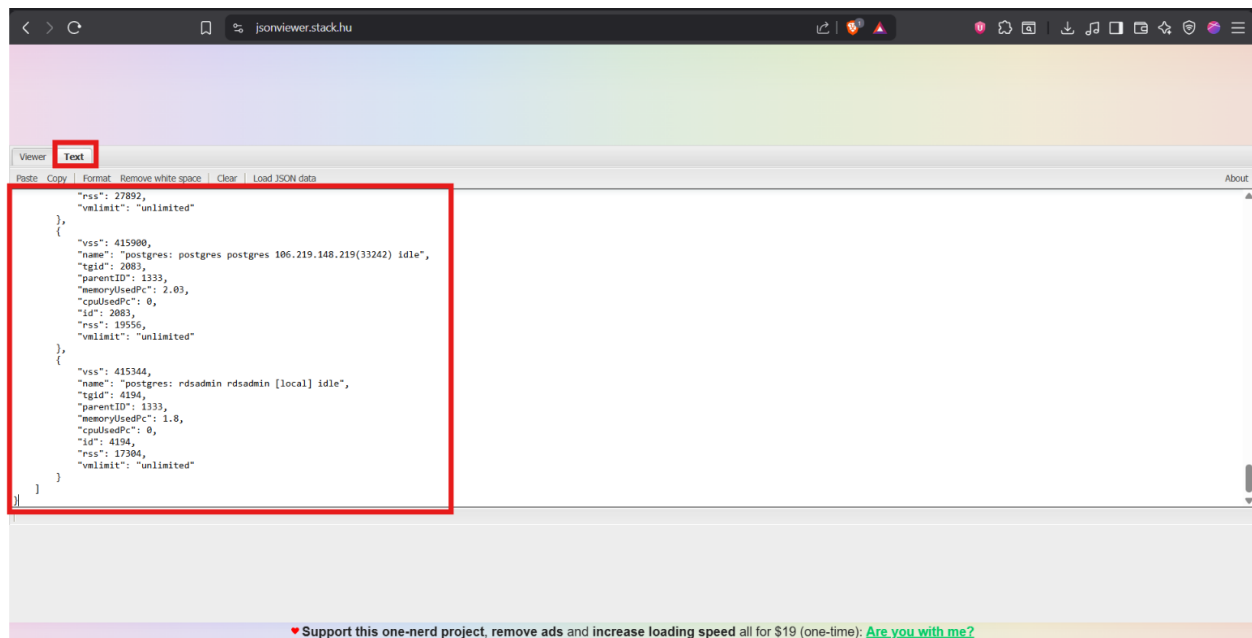
Filter events - press enter to search

Timestamp	Message
2026-06-20T13:16:09.000Z	{"engine":"POSTGRES","instanceID":"database-1","instanceResourceID":"db-SQUGNWX4C2JEMH5CZVUGCMFY","timestamp":"2026-06-20T13:16:09Z","version":1,"uptime":1.0}
2026-06-20T13:16:39.000Z	{"engine":"POSTGRES","instanceID":"database-1","instanceResourceID":"db-SQUGNWX4C2JEMH5CZVUGCMFY","timestamp":"2026-06-20T13:16:39Z","version":1,"uptime":1.0}
2026-06-20T13:17:09.000Z	{"engine":"POSTGRES","instanceID":"database-1","instanceResourceID":"db-SQUGNWX4C2JEMH5CZVUGCMFY","timestamp":"2026-06-20T13:17:09Z","version":1,"uptime":1.0}
2026-06-20T13:17:39.000Z	<pre>{ "engine": "POSTGRES", "instanceID": "database-1", "instanceResourceID": "db-SQUGNWX4C2JEMH5CZVUGCMFY", "timestamp": "2026-06-20T13:17:39Z", "version": 1, "uptime": "01:18:46", "numCPUs": 2, "cpuUtilization": { "guest": 0, "irq": 0, "system": 1.1, "wait": 0.1, "idle": 97.1, "user": 1.7, "total": 2.9, "steal": 0, "nice": 0 }, "loadAverageMinute": { "one": 0.1, "five": 0.1, "fifteen": 0.08 } }</pre>

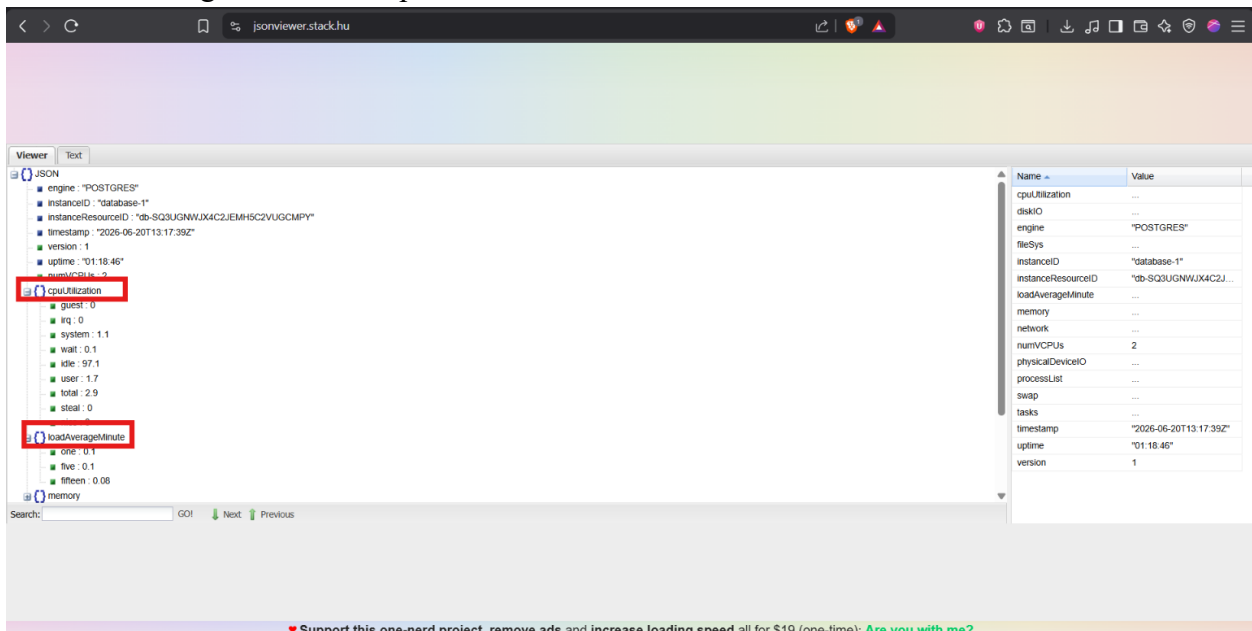
[Copy](#)

[Back to top](#)

- Here we will paste it on this site <https://jsonviewer.stack.hu/>.



- Switch to **Viewer** and we can see the categories it was generating logs for like loadAverageMinute and cpuUtilization.



Verification

- Amazon RDS Enhanced Monitoring metrics were reviewed successfully.
- A custom CloudWatch dashboard was created successfully.
- RDS metrics were added to the dashboard.
- Enhanced Metrics JSON was also verified with its proper timestamp and data generation.